

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Homework 10: due on 4/11
  - Project 3: due on 4/11
  - Lab 4: due on 4/15

Previous Lecture …

- Indexed Allocation
- Directory contents
- Free block tracking
  - bitmap
  - linked list of free blocks

Muddiest Points (1/3)

- Bitmapping for free blocks
- It seems that directories have a limit to the number of files / subdirectories they contain based on the size of a disk block. Is / how is this limit overcome?
- Everything
- are the index nodes for the directories stored on the heap? is this different than the index nodes from the file allocation table?
- Variable sized file name structures' pros and cons
- what exactly is an i-node and where is it stored?
- Might need some recap on inode. So inode contains information for a specific file and block index is a block storing only the addresses of all the file block?

Muddiest Points (2/3)

- a little confused about Fast File System indexing
- For FFS, all the files will be kept in one directoriy (or many directories?) and then we adopt the direct pointers or one-level or multi-level pointers to point to their data based on their size?
- Typically one directory per index block? What about a subdirectory in a parent directory? Would there be another block index (for the subdirectory) stored in the parent directory?
- Can you explain why smaller page size leads to more page fault?
- Linked allocation versus contiguous and when is it optimal to use linked

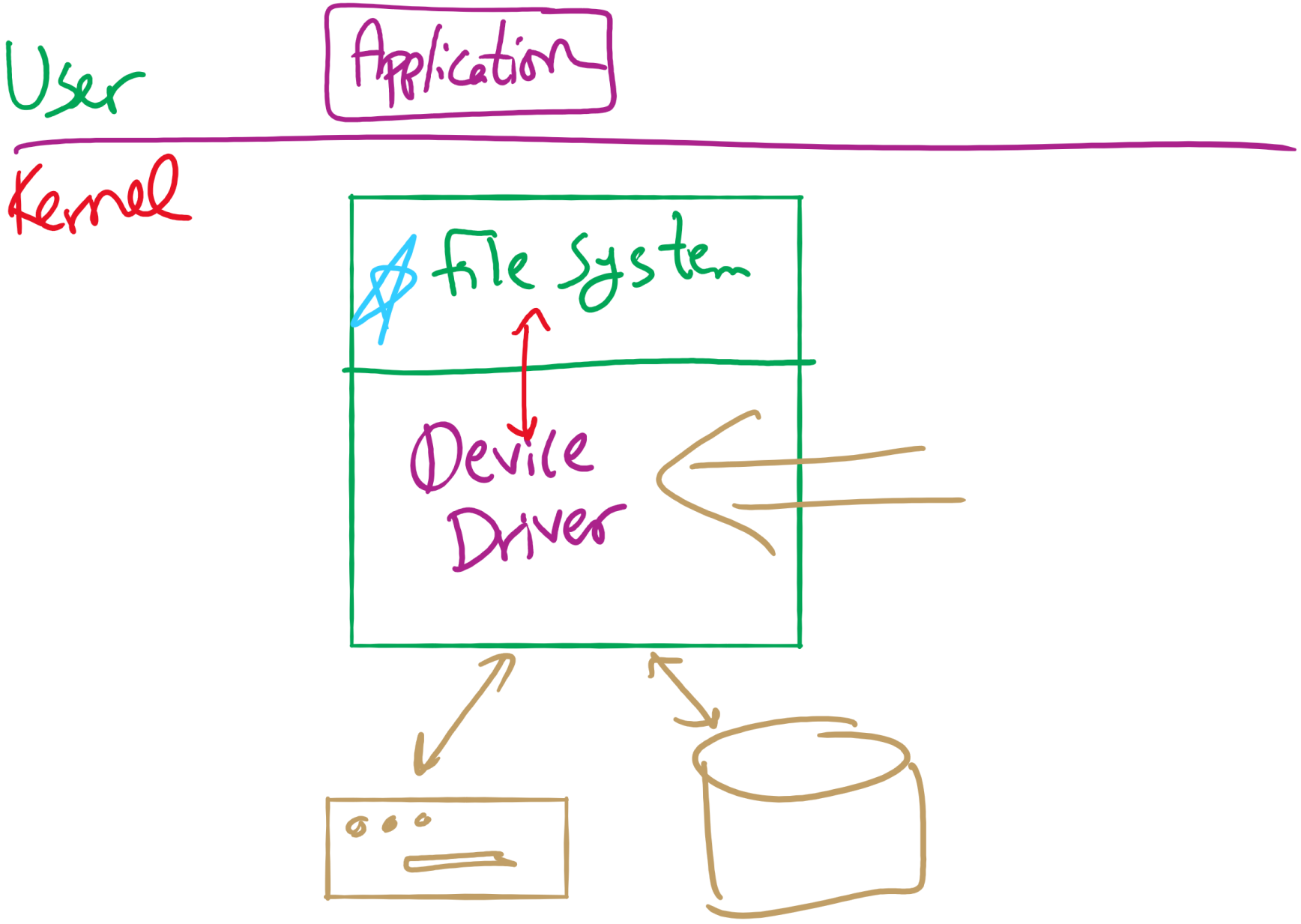
Muddiest Points (3/3)

- how do we do midterm corrections
- Top hat question about partition
- Still a but confused on indexing for the Page Table Allocation
- structure of a disk section

Problem of the Day – Part 1

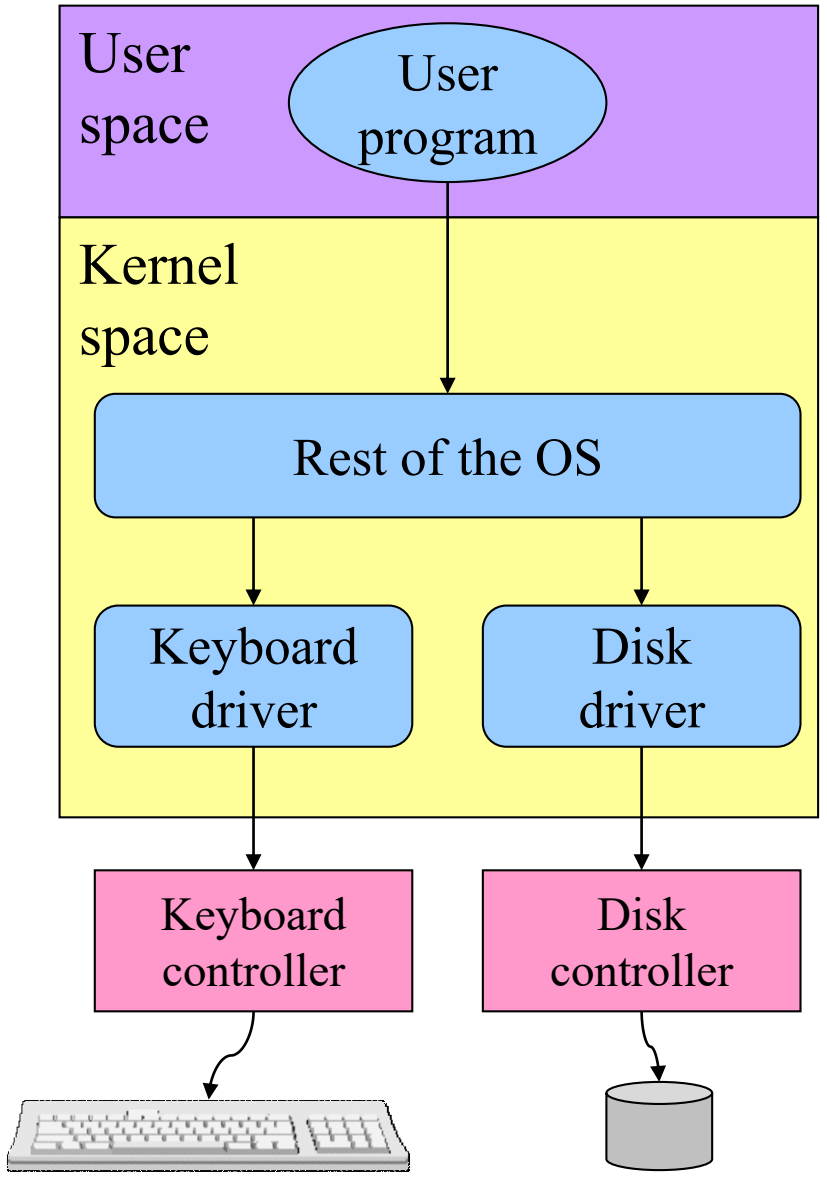
- How does a file system handle errors?
- Answer: Defense in Depth
  - multiple layers of error detection/correction

Software Stack

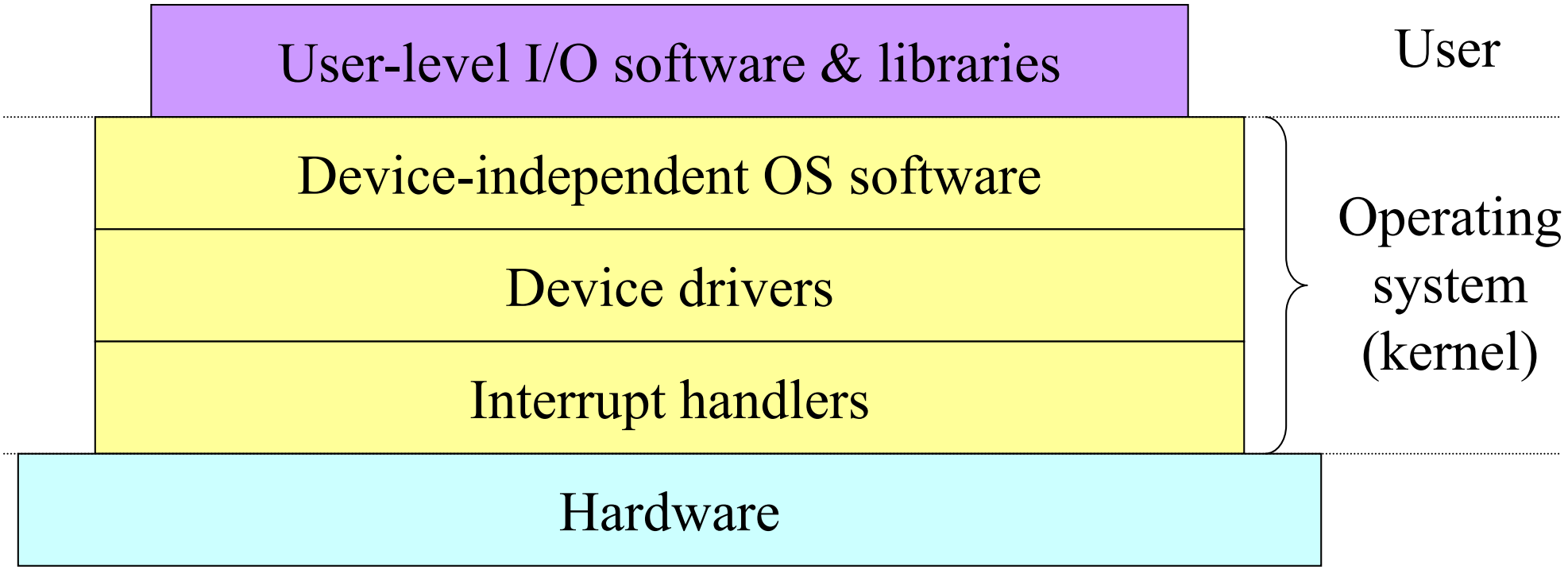


Device drivers

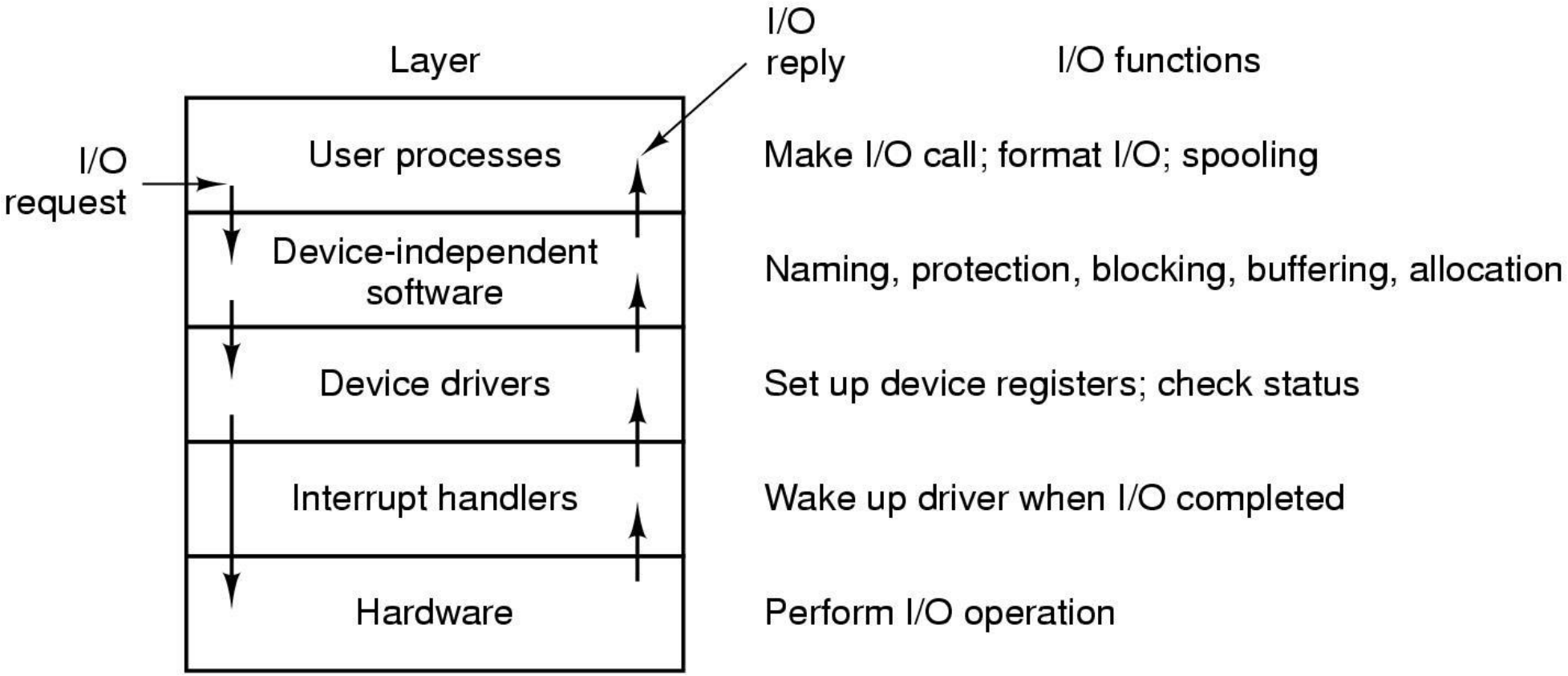
- Device drivers go between device controllers and rest of OS
  - Drivers standardize interface to widely varied devices
- Device drivers communicate with controllers over bus
  - Controllers communicate with devices themselves



Layers of I/O software



Anatomy of an I/O request

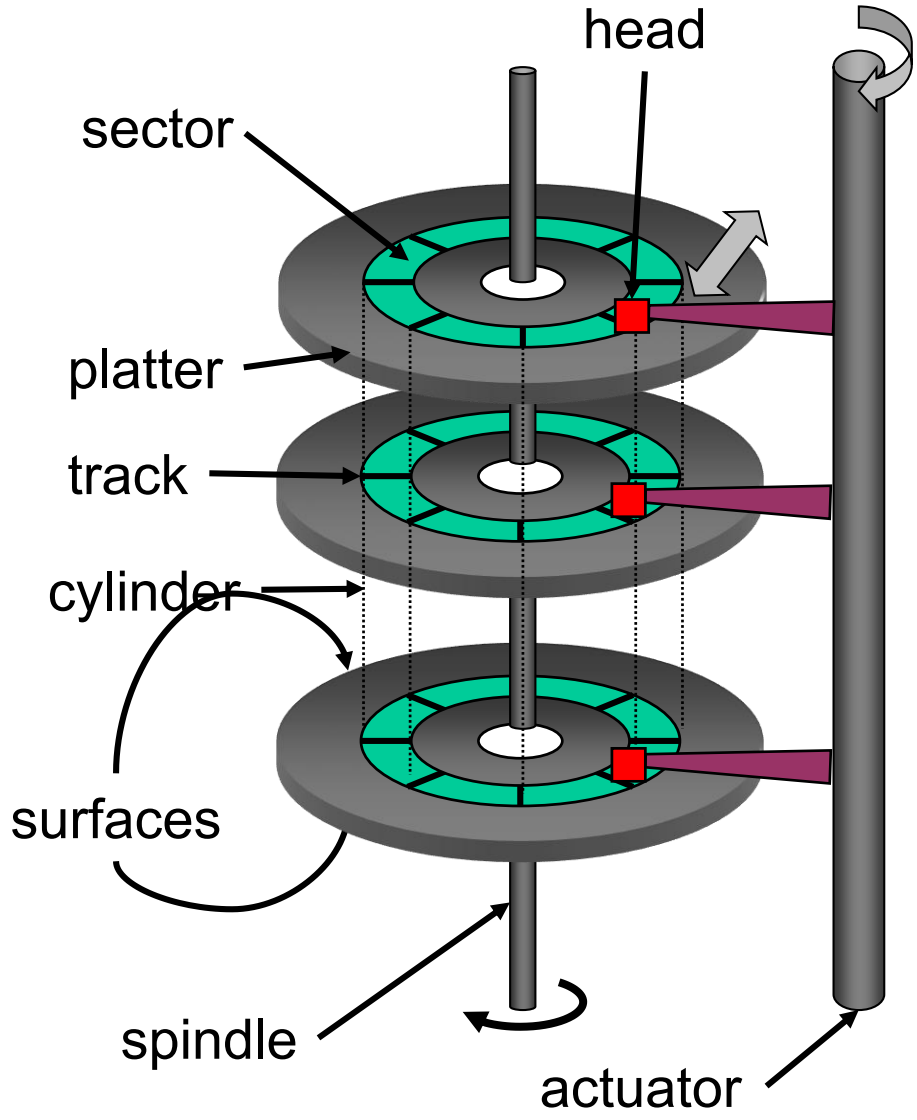


Device Driver goals

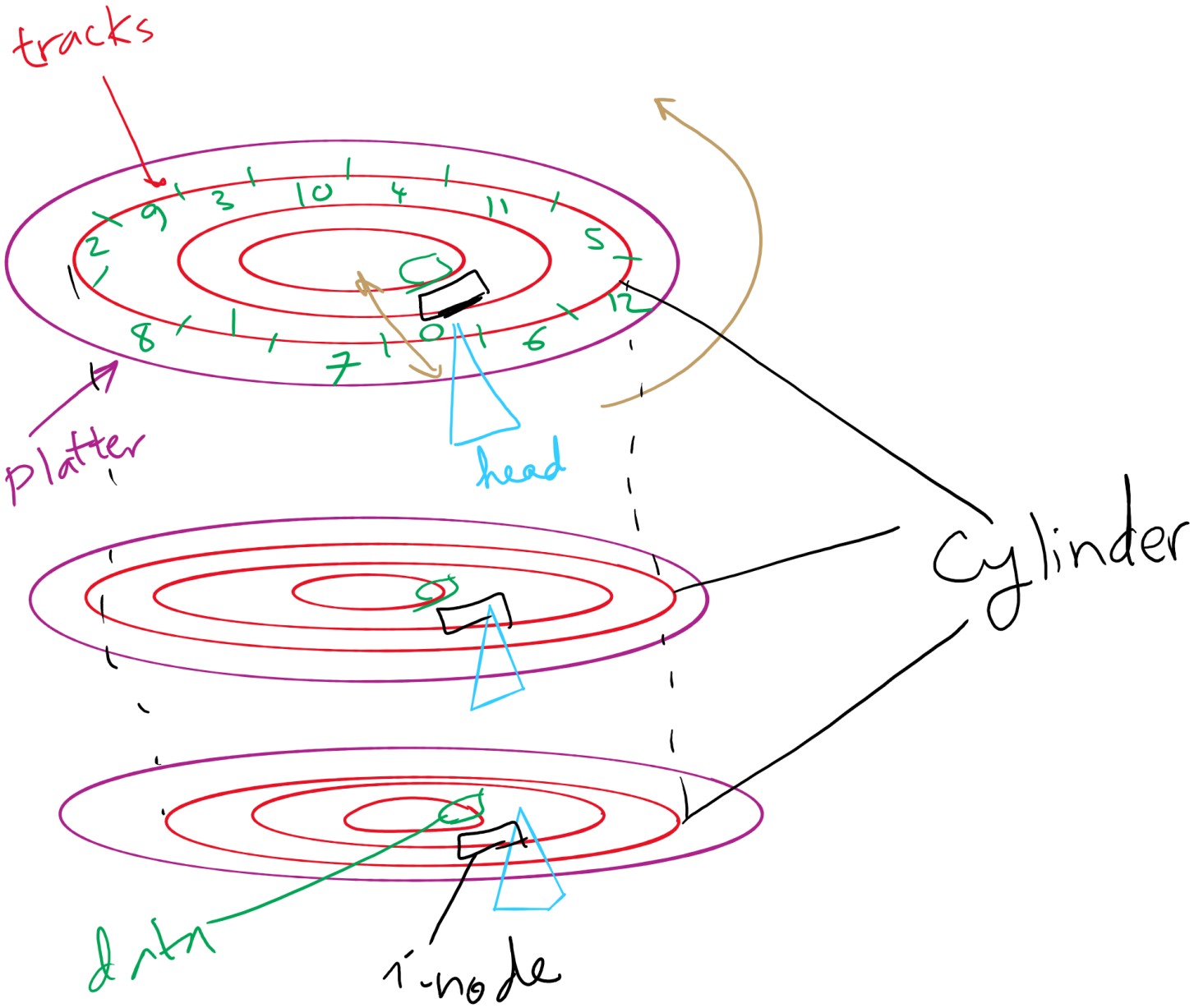
- Device independence
  - Programs can access any I/O device
  - No need to specify device in advance
- Uniform naming
  - Name of a file or device is a string or an integer
  - Doesn't depend on the machine (underlying hardware)
- **Error handling**
  - **Done as close to the hardware as possible**
  - **Isolate from higher-level software**
- Synchronous vs. asynchronous transfers
  - Blocked transfers vs. interrupt-driven
- Buffering
  - Data coming off a device cannot be stored in final destination
- Sharable vs. dedicated devices

Disk drive structure

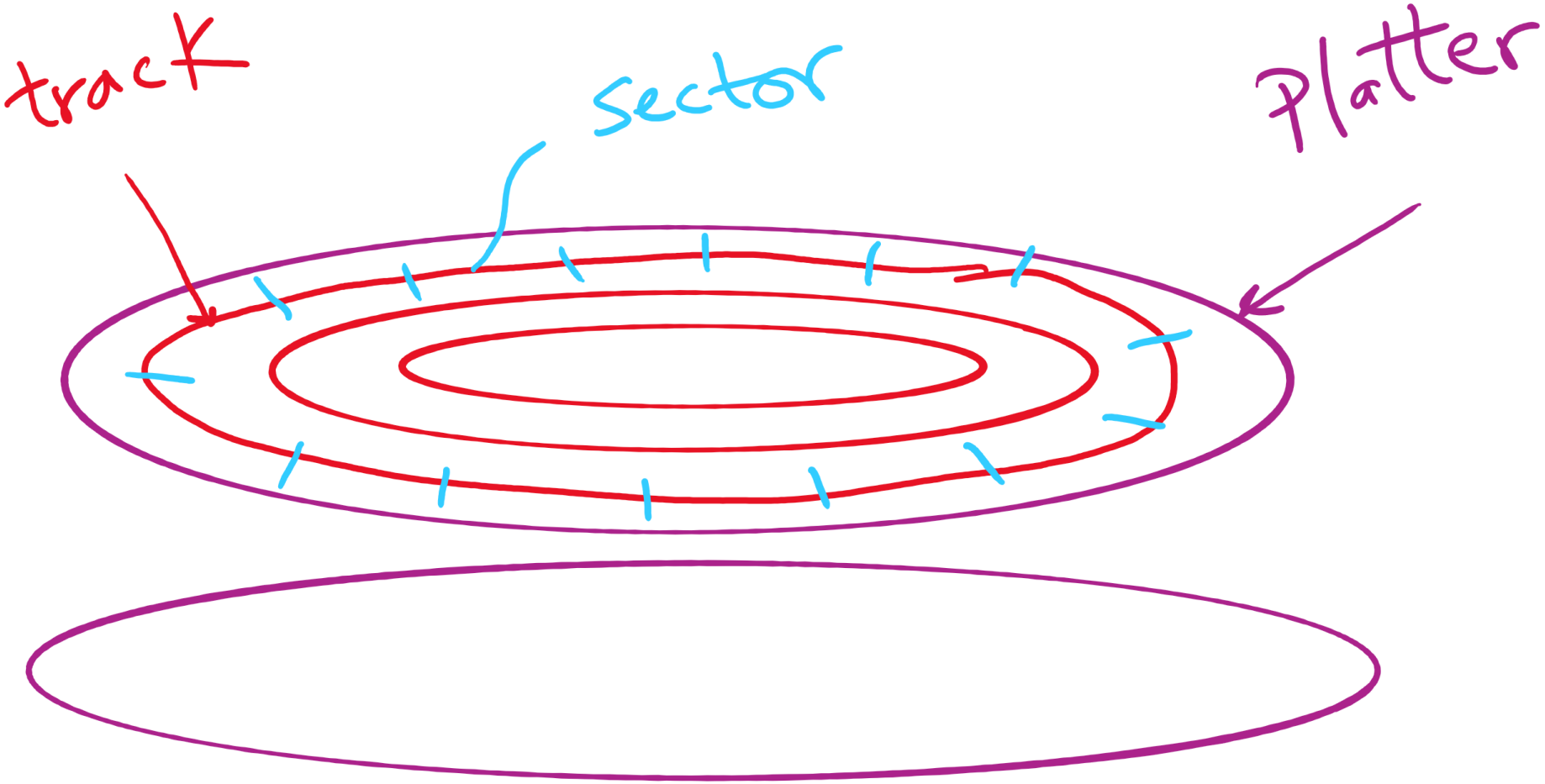
- Data stored on surfaces
  - Up to two surfaces per platter
  - One or more platters per disk
- Data in concentric tracks
  - Tracks broken into sectors
    - 256B-1KB per sector
  - Cylinder: corresponding tracks on all surfaces
- Data read and written by heads
  - Actuator moves heads
  - Heads move in unison



Disks, cylinders, cylinder groups



Disk Sector



What's in a disk request?

- Time required to read or write a disk block determined by 3 factors
  - Seek time
  - Rotational delay
    - Average delay = 1/2 rotation time
    - Example: rotate in 10ms, average rotation delay = 5ms
  - Actual transfer time
    - Transfer time = time to rotate over sector
    - Example: rotate in 10ms, 200 sectors/track => 10/200 ms = 0.05ms transfer time per sector
- Seek time dominates, with rotation time close
- Error checking is done by controllers

Disk drive specifics

	IBM 360KB floppy	WD 18GB HD
Cylinders	40	10601
Tracks per cylinder	2	12
Sectors per track	9	281 (average)
Sectors per disk	720	35742000
Bytes per sector	512	512
Capacity	360 KB	18.3 GB
Seek time (minimum)	6 ms	0.8 ms
Seek time (average)	77 ms	6.9 ms
Rotation time	200 ms	8.33 ms
Spinup time	250 ms	20 sec
Sector transfer time	22 ms	17 μsec

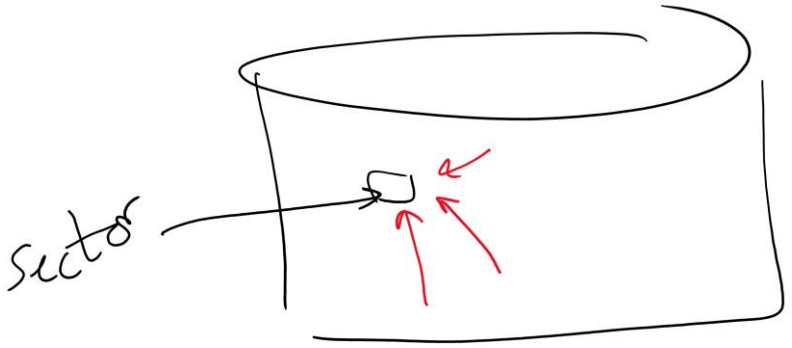
$$\text{Sectors/disk} = \frac{\text{Sectors}}{\text{track}} * \frac{\text{tracks}}{\text{cylinder}} * \frac{\text{cylinders}}{\text{disk}}$$

$$\text{disk capacity (bytes/disk)} = \frac{\text{bytes}}{\text{sector}} * \frac{\text{sectors}}{\text{disk}}$$

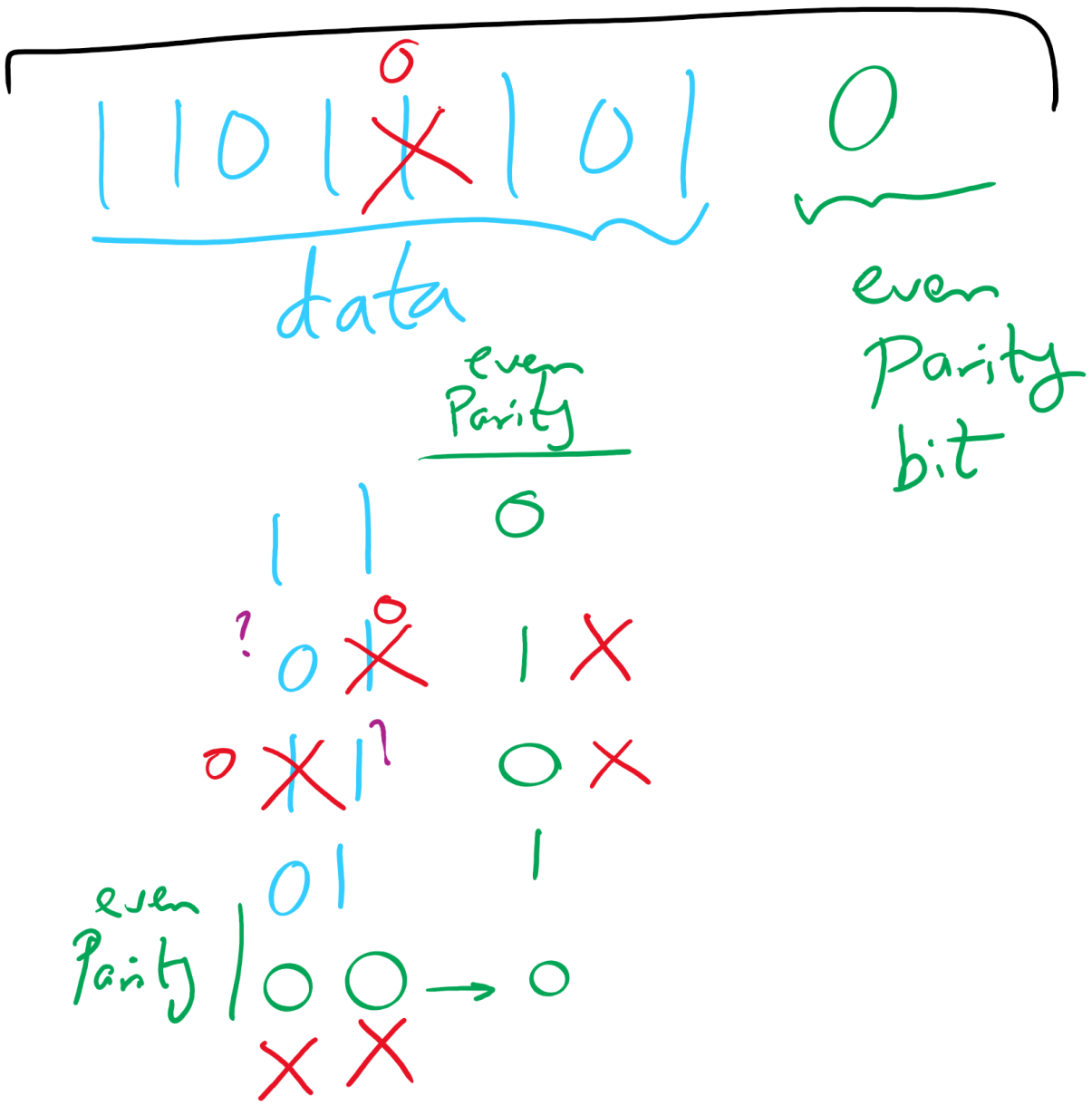
Structure of a disk sector

- Preamble contains information about the sector
  - Sector number & location information
- Data is usually 256, 512, or 1024 bytes
- ECC (Error Correcting Code) is used to detect & correct minor errors in the data

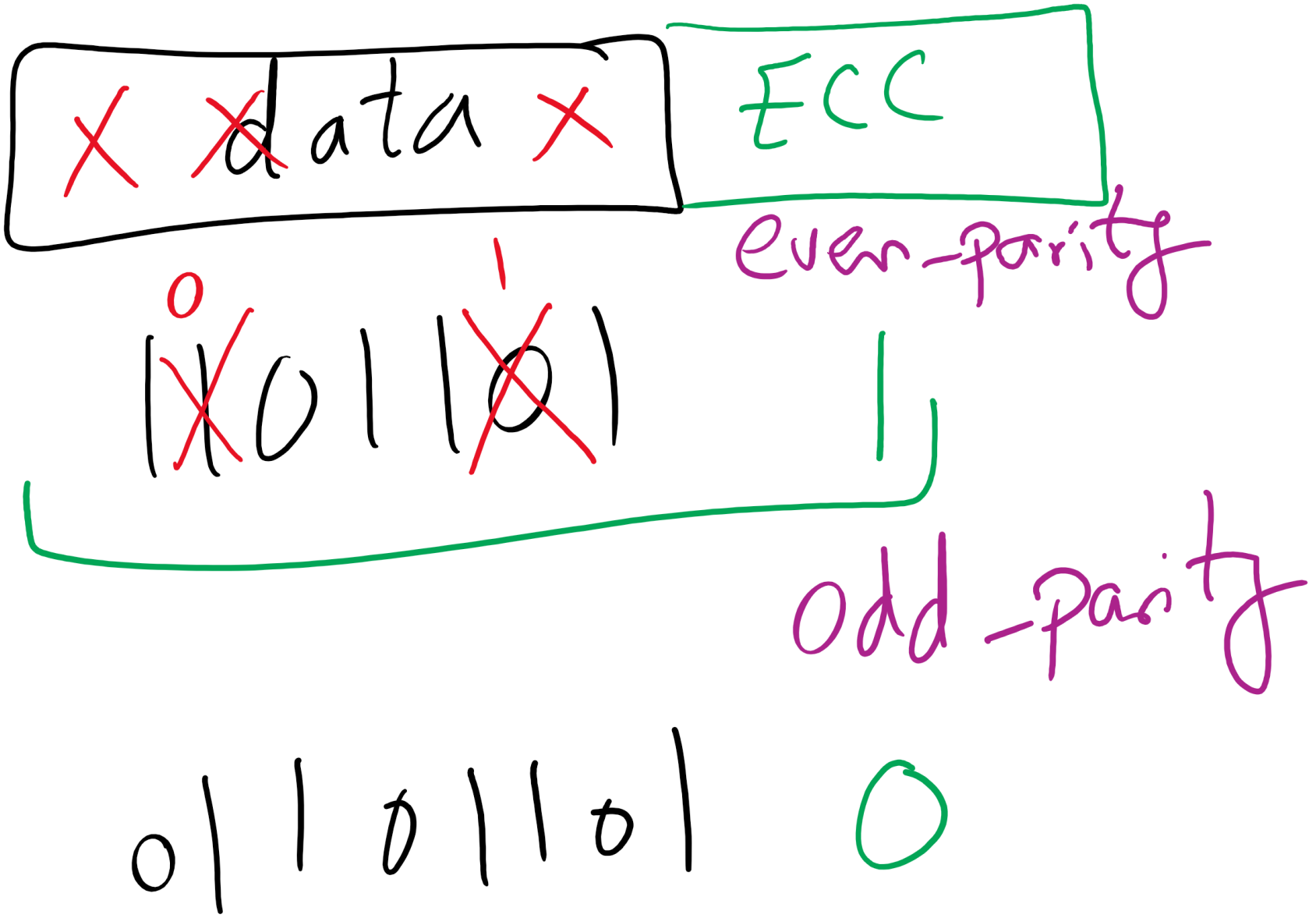




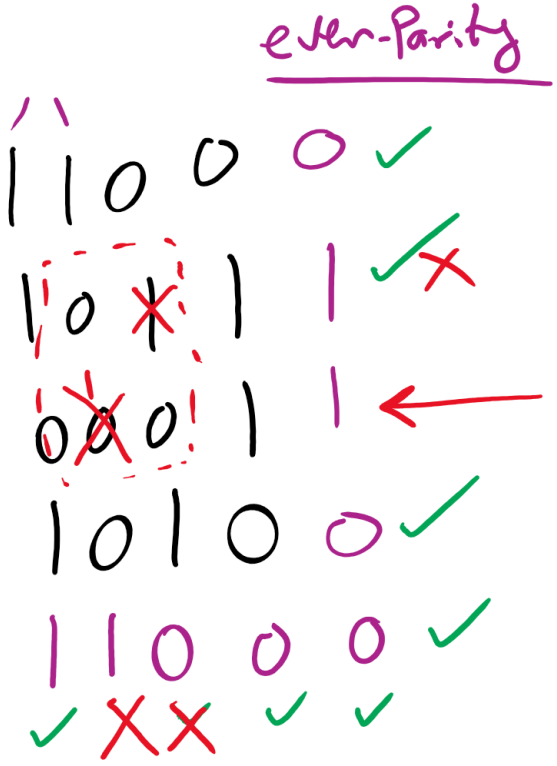
Parity Examples

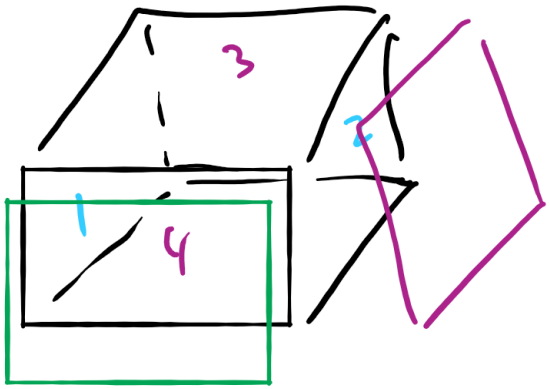


1-d parity



2-d parity


even-Parity
1 1 0 0 0
1 0 1 1 1
0 0 0 1 1
1 0 1 0 0
1 1 0 0 0
Detect: 2-bit errors
Correct: 1-bit errors


3
2
1
4

When good disks go bad…

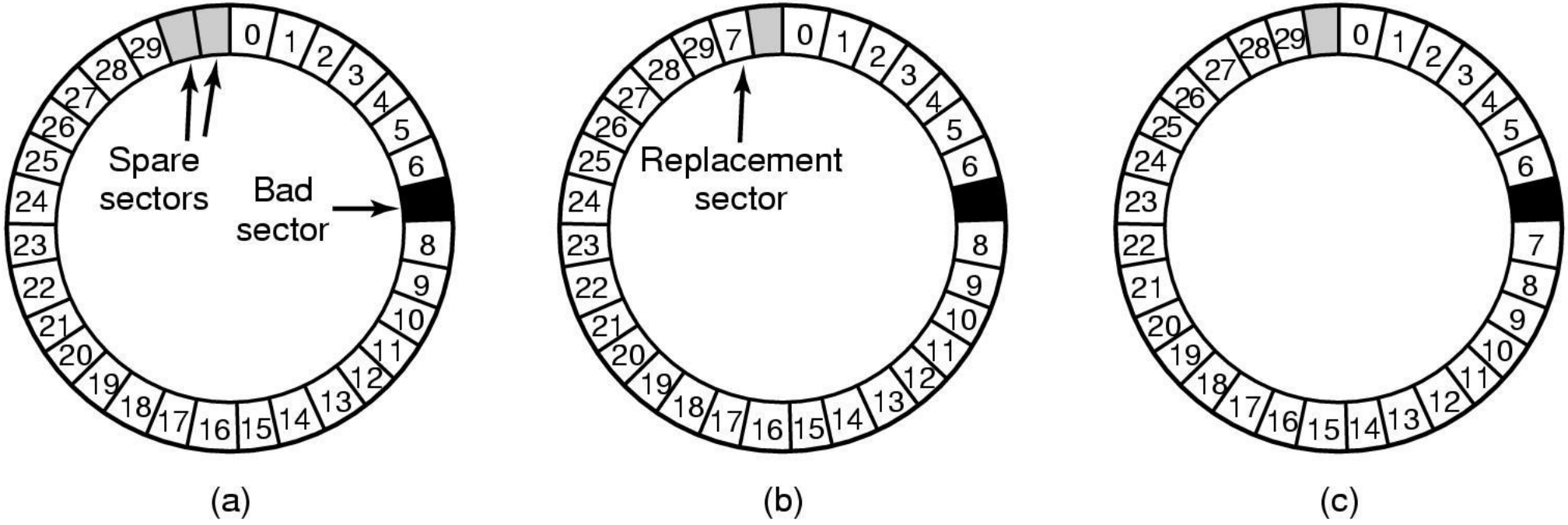
Disks have defects

In 3M+ sectors, this isn't surprising!

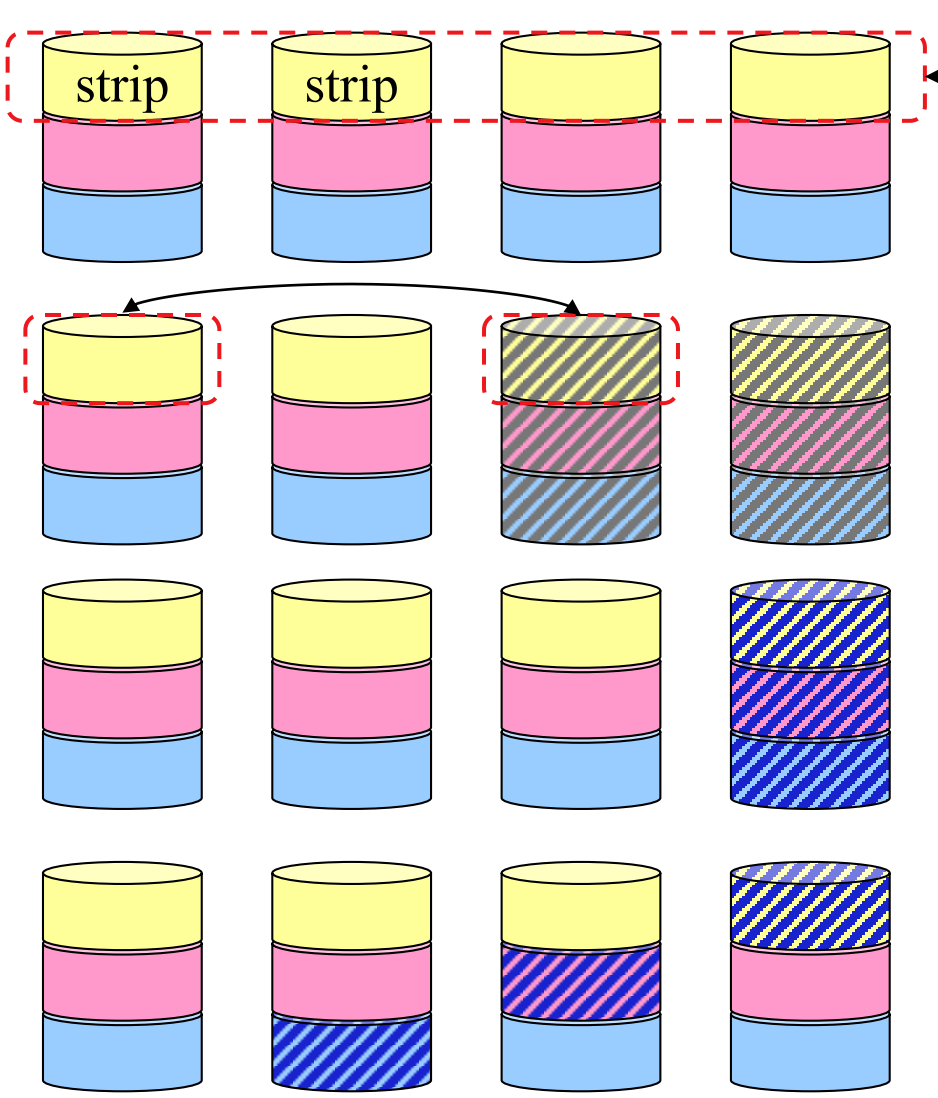
ECC helps with errors, but sometimes this isn't enough

Disks keep spare sectors (normally unused) and remap bad sectors into these spares

If there's time, the whole track could be reordered…



RAIDs, RAIDs, and more RAIDs



RAID 0
(Redundant Array of Inexpensive Disks

RAID 1
(Mirrored copies)

RAID 4
(Striped with parity)

RAID 5
(Parity rotates through disks)